

Cross-Attention in Transformers

Mechanism, Mathematics, and Applications

Introduction to Deep Learning

April 24, 2026

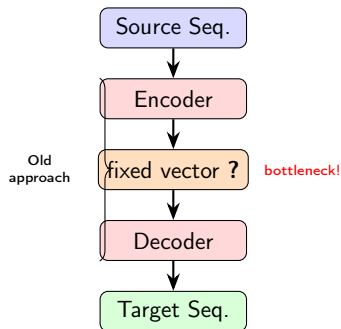
Outline

- 1 Motivation & Background
- 2 Mathematical Formulation
- 3 Architecture: Cross-Attention
- 4 Masked Self-Attention in the Decoder
- 5 Intuition & Visualisation
- 6 Computational Considerations
- 7 Variants & Extensions
- 8 Summary

What Problem Does Cross-Attention Solve?

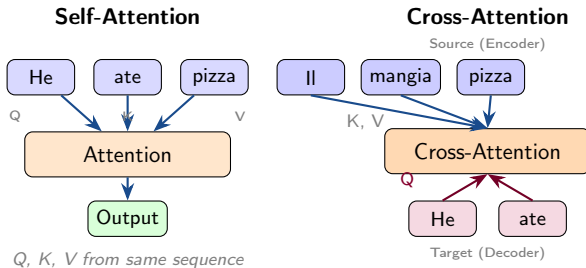
The Encoder-Decoder Challenge

- Many tasks require **mapping one sequence to another** (translation, summarisation, speech recognition)
- Encoder produces a rich contextual representation of the **source**
- Decoder must selectively attend to **relevant parts** of the source while generating the **target**
- A fixed-length bottleneck vector loses information for long sequences



Instead of compressing the source to a single vector, let the decoder *query* the encoder at every step.

Self-Attention vs. Cross-Attention



Self-Attention

Q, K, V all come from the **same** sequence.
Used in encoder and decoder (masked).

Cross-Attention

Q from **decoder**, K and V from **encoder output**. Bridges two different sequences.

Scaled Dot-Product Attention (Recap)

General Attention Formula (Vaswani et al., 2017)

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

Tensor shapes:

- $\mathbf{Q} \in \mathbb{R}^{T_q \times d_k}$ — query matrix
- $\mathbf{K} \in \mathbb{R}^{T_k \times d_k}$ — key matrix
- $\mathbf{V} \in \mathbb{R}^{T_k \times d_v}$ — value matrix
- Output $\in \mathbb{R}^{T_q \times d_v}$

Why scale by $1/\sqrt{d_k}$?

- Prevents dot products from growing very large
- Keeps softmax in a well-behaved gradient region

Scores Matrix

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \in \mathbb{R}^{T_q \times T_k}$$

S_{ij} = relevance of source token j to query token i .

Attention Weights

$$\mathbf{A} = \text{softmax}_{\text{row}}(\mathbf{S}) \in \mathbb{R}^{T_q \times T_k}$$

Cross-Attention: Precise Definition

Let $X \in \mathbb{R}^{T_x \times d_{\text{model}}}$ be the **encoder output** and $Y \in \mathbb{R}^{T_y \times d_{\text{model}}}$ be the **decoder hidden state**.

Step 1 — Linear projections:

$$Q = YW^Q, \quad K = XW^K, \quad V = XW^V$$

where $W^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$.

Step 2 — Compute cross-attention:

$$\text{CrossAttn}(X, Y) = \text{softmax}\left(\frac{(YW^Q)(XW^K)^T}{\sqrt{d_k}}\right) (XW^V)$$

Key Asymmetry

- The **query** length is T_y (target); the **key/value** length is T_x (source)
- $T_x \neq T_y$ in general — cross-attention handles variable-length pairs naturally

Step-by-Step Computation

1. Project to Q, K, V

For each decoder position i , encoder position j :

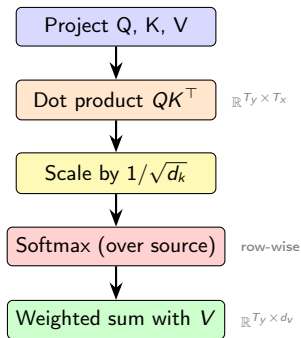
$$q_i = y_i W^Q, \quad k_j = x_j W^K, \quad v_j = x_j W^V$$

2. Compute attention score

$$e_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

3. Normalise over source positions

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'=1}^{T_x} \exp(e_{ij'})}$$



Multi-Head Cross-Attention

Motivation

Different heads attend to **different aspects** of the source (syntax, semantics, position).

For each head $h = 1, \dots, H$:

$$\text{head}_h = \text{Attention}(Y W_h^Q, X W_h^K, X W_h^V), \quad W_h^Q, W_h^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_h^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

$$\text{MultiHead}(X, Y) = [\text{head}_1 \parallel \dots \parallel \text{head}_H] W^O, \quad W^O \in \mathbb{R}^{H d_v \times d_{\text{model}}}$$

Typical Hyperparameters (base model)

Quantity	Value
d_{model}	512
H (heads)	8
$d_k = d_v$	64
$H \times d_k = 512 = d_{\text{model}} \quad \checkmark$	

Parameter Count

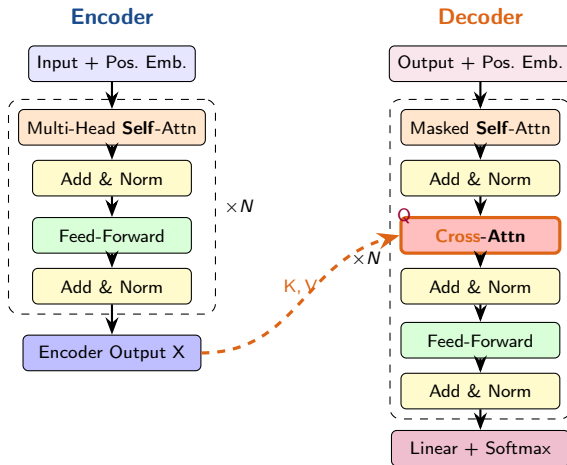
Per head: $3 \times d_{\text{model}} \times d_k$

Total Q, K, V: $3 \times d_{\text{model}}^2$

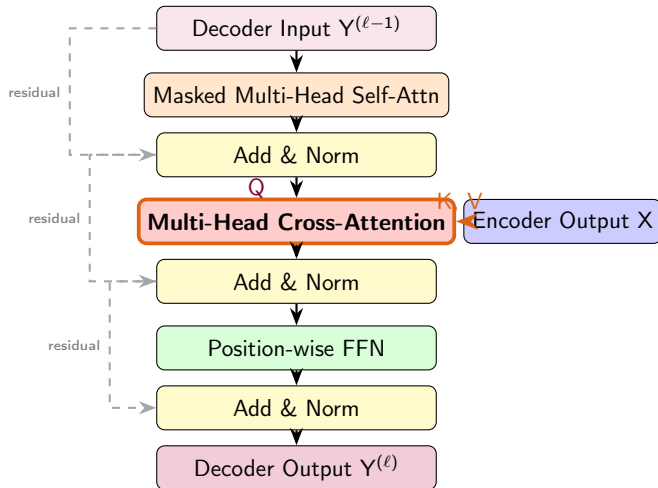
Output projection: d_{model}^2

Grand total: $4 d_{\text{model}}^2$

The Original Transformer Architecture



Inside a Decoder Layer



The Residual + LayerNorm Wrapper

Each sublayer (including cross-attention) is wrapped identically:

$$Y^{\text{out}} = \text{LayerNorm}(Y^{\text{in}} + \text{MultiHeadCrossAttn}(X, Y^{\text{in}}))$$

Residual Connection

- Allows gradients to flow directly through layers
- Enables training of very deep networks
- Output = input + sublayer transformation

Layer Normalisation

$$\text{LN}(h) = \frac{h - \mu}{\sigma + \varepsilon} \gamma + \beta$$

Normalises over the *feature* dimension per token. Stabilises training.

Pre-LN vs. Post-LN

- **Post-LN** (original paper): LN after residual addition — can be unstable early in training
- **Pre-LN** (GPT-2 and later): LN *before* sublayer — more stable, now standard

Why the Decoder Needs a Mask

At training time, the full target sequence is fed to the decoder all at once (for parallelism). But token i must **only** depend on tokens $1, \dots, i - 1$ — not on future tokens it hasn't generated yet.

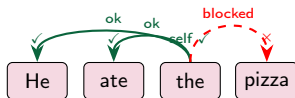
Without a mask...

Token 3 could directly attend to token 5, “seeing the answer” before predicting it. The model would learn a trivial copy task rather than genuine language modelling.

Solution: Causal (look-ahead) mask

Set all attention scores *to the future* to $-\infty$ before the softmax. After softmax those weights become exactly 0.

Attention from “the”



Can see past; cannot see future

The Causal Mask: Construction & Effect

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V$$

where the causal mask $M \in \mathbb{R}^{T \times T}$ is:

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

Effect: $\exp(-\infty) = 0$, so after softmax the future positions receive **exactly zero weight**.

Causal mask M for $T = 4$:

	$j=1$	$j=2$	$j=3$	$j=4$
$i=4$	0	$-\infty$	$-\infty$	$-\infty$
$i=3$	0	0	$-\infty$	$-\infty$
$i=2$	0	0	0	$-\infty$
$i=1$	0	0	0	0
	allowed	allowed	allowed	blocked

In practice

$-\infty$ is implemented as a large negative number (e.g. -10^9 in PyTorch) to avoid NaN from true infinity.

Three Attention Types: Side-by-Side

		Q from	K, V from	Mask
Encoder Attn	Self-	Encoder tokens	Encoder tokens	None — sees full source freely
Decoder Self-Attn	Masked	Decoder tokens	Decoder tokens	Causal — token i sees only positions $1 \dots i$
Decoder Attn	Cross-	Decoder tokens	Encoder tokens	Padding mask only (optional) — sees all source positions

Common Misconception

"Cross-attention is masked" — it is **not** causally masked. The causal mask lives exclusively in the *decoder self-attention* sublayer.

Why cross-attention needs no causal mask

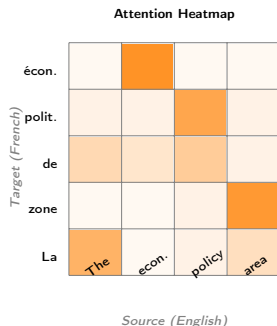
The encoder output X is a *complete*, fixed sequence. There are no "future" encoder tokens to hide — the decoder may look at all of them simultaneously.

What Do the Attention Weights Mean?

Recall: weight α_{ij} captures how much **decoder token i** attends to **encoder token j** .

Machine Translation Example (English $\rightarrow$ French):

- Generating “zone” $\rightarrow$ high attention on “area”
- Generating *économique* $\rightarrow$ high attention on “economic”
- Adjective–noun reordering handled automatically

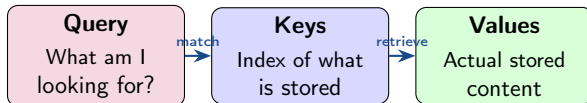


Interpretability

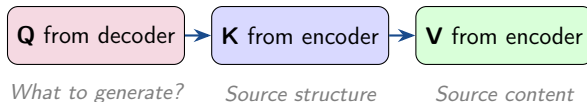
Attention maps provide a window into what the model focuses on — valuable for debugging and analysis.

Query–Key–Value Analogy

Database Analogy



In Cross-Attention



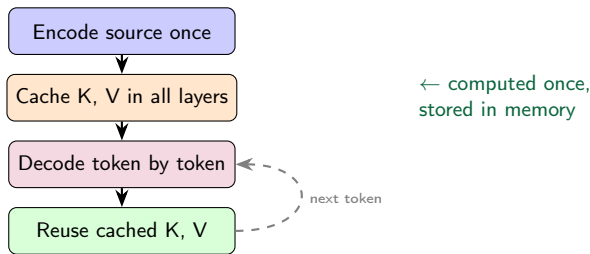
Intuition

The decoder asks: “Given what I want to generate (Q), which parts of the source (K) are most relevant, and what information do I retrieve from them (V)?”

KV-Caching at Inference

Why it matters

During autoregressive decoding, the **encoder output X is fixed**. We only need $K = XW^K$ and $V = XW^V$ **once per source**.



Speedup: Reduces per-step cross-attention cost from $\mathcal{O}(T_x d^2)$ to $\mathcal{O}(T_x d)$ (only the Q projection and matmul are repeated).

Cross-Attention Beyond NLP

Domain	Source (K, V from)	Query (Q from)
Machine Translation	Source sentence	Partial translation
Speech Recognition	Audio encoder	Text decoder
Image Captioning	Vision encoder	Caption decoder
Text-to-Image	Text encoder	Diffusion UNet
Multimodal LLMs	Image/video tokens	Language tokens
Perceiver IO	High-dim inputs	Latent array
Robotics (RT-2)	Language goal	Action decoder

General Principle

Whenever you need to **condition one modality or sequence on another**, cross-attention provides a flexible, differentiable alignment mechanism.

Core ideas:

- Cross-attention lets the **decoder** selectively retrieve information from the **encoder**
- **Q** comes from the decoder; **K**, **V** come from the encoder output
- Generalises to any source $\rightarrow$ target alignment problem
- Handles variable-length pairs with no additional mechanism

Key properties:

- $\mathcal{O}(T_y T_x d)$ time complexity
- Fully parallel (no recurrence)
- KV-caching enables efficient autoregressive decoding
- Multi-head captures diverse alignment patterns

Applications

MT, ASR, image captioning, text-to-image, multimodal LLMs, robotics, ...